

1. Types of Operators in C#

C# has the following operator categories:

Arithmetic Operators — perform math operations.

csharp

```
int a = 10, b = 3;
Console.WriteLine(a + b); // 13 (Addition)
Console.WriteLine(a - b); // 7 (Subtraction)
Console.WriteLine(a * b); // 30 (Multiplication)
Console.WriteLine(a / b); // 3 (Division - integer)
Console.WriteLine(a % b); // 1 (Modulus - remainder)
```

Relational (Comparison) Operators — compare two values, return bool.

csharp

```
Console.WriteLine(a > b); // True
Console.WriteLine(a < b); // False
Console.WriteLine(a >= 10); // True
Console.WriteLine(a == b); // False
Console.WriteLine(a != b); // True
```

Logical Operators — combine boolean expressions.

csharp

```
bool x = true, y = false;
Console.WriteLine(x && y); // False (AND - both must be true)
Console.WriteLine(x || y); // True (OR - at least one true)
Console.WriteLine(!x); // False (NOT - reverses)
```

Bitwise Operators — operate on individual bits.

csharp

```
int p = 5; // 0101 in binary
int q = 3; // 0011 in binary
Console.WriteLine(p & q); // 1 (AND)
Console.WriteLine(p | q); // 7 (OR)
Console.WriteLine(p ^ q); // 6 (XOR)
Console.WriteLine(~p); // -6 (NOT)
Console.WriteLine(p << 1); // 10 (Left shift)
Console.WriteLine(p >> 1); // 2 (Right shift)
```

Assignment Operators — assign and optionally compute.

csharp

```
int n = 10;
n += 5; // n = 15 (same as n = n + 5)
n -= 3; // n = 12
n *= 2; // n = 24
n /= 4; // n = 6
n %= 4; // n = 2
```

Miscellaneous / Special Operators:

csharp

```
// Ternary operator
int age = 20;
string result = (age >= 18) ? "Adult" : "Minor";

// typeof
Console.WriteLine(typeof(int)); // System.Int32

// sizeof
Console.WriteLine(sizeof(int)); // 4 (bytes)

// is operator (type check)
object obj = "Hello";
Console.WriteLine(obj is string); // True
```

2. Value Type vs Reference Type — with Code Examples

(Already in the document as Q5, but here's an extended version with code)

```
csharp
// — VALUE TYPE EXAMPLE —
int a = 10;
int b = a;    // full copy of value
b = 50;
Console.WriteLine(a); // 10 — unchanged, completely independent

// — REFERENCE TYPE EXAMPLE —
class Person { public string Name; }

Person p1 = new Person();
p1.Name = "Ram";

Person p2 = p1;    // copies the REFERENCE, not the object
p2.Name = "Shyam";

Console.WriteLine(p1.Name); // "Shyam" — p1 is also affected!
// Both p1 and p2 point to the SAME object in heap
```

Feature	Value Type	Reference Type
Storage	Stack	Heap (reference on stack)
Assignment	Copies value	Copies reference
Independence	Changes independent	Changes shared
Examples	int, float, bool, char, struct, enum	class, string, array, delegate
Default value	0 / false / '\0'	null

3. Execution of a .NET Application

(Covered in L1 of the document, but here's a quick summary)

```
C# Source Code (.cs)
    ↓ [C# Compiler - csc.exe]
Assembly: MSIL + Metadata (.exe / .dll)
    ↓ [CLR loads the Assembly]
JIT Compiler converts MSIL → Native Machine Code
    ↓
Operating System executes the native code
```

Key points: MSIL is CPU-agnostic. JIT runs platform-specific optimization. CLR provides GC, security, and exception handling throughout execution.

4. Components of .NET Framework — CLR, CLS, FCL, BCL

CLR (Common Language Runtime) The core runtime engine. Manages execution, memory (GC), security, and exception handling. Every .NET program runs inside the CLR.

CLS (Common Language Specification) A set of rules and standards that all .NET languages must follow to ensure interoperability. For example, both VB.NET's `Integer` and C#'s `int` compile down to the same `System.Int32` — this is CLS at work. It enables code written in C# to be used from VB.NET and vice versa.

FCL (Framework Class Library) A huge collection of pre-built, reusable classes covering everything: I/O, networking, XML, collections, threading, database access, UI frameworks (WinForms, WPF), web (ASP.NET), etc. Developers use these instead of writing everything from scratch.

BCL (Base Class Library) A subset of FCL — the absolute foundational core. Includes namespaces like `System`, `System.IO`, `System.Collections`, `System.Text`. BCL is what every .NET app needs at minimum. Think of FCL as the full library and BCL as the essential foundation inside it.

```
FCL (Full Library)
├─ BCL (Core Foundation: System, IO, Collections...)
└─ ASP.NET, WPF, ADO.NET, LINQ...
```


5. Language Interoperability

Language interoperability means that code written in different .NET languages (C#, VB.NET, F#) can work together seamlessly in the same application.

How it works: All .NET languages compile down to the same MSIL (Intermediate Language). The CLS defines common rules so every language produces compatible MSIL. The CLR then executes all of it uniformly.

Example:

csharp

```
// MathLib.vb (VB.NET Library)
Public Class Calculator
    Public Function Add(a As Integer, b As Integer) As Integer
        Return a + b
    End Function
End Class
```

csharp

```
// Program.cs (C# Project - uses the VB.NET class)
Calculator calc = new Calculator();
int result = calc.Add(5, 3); // works perfectly!
Console.WriteLine(result);    // 8
```

Modern examples from your notes:

- Python.NET (pythonnet) — Python calling C# code
- C# & C++ via P/Invoke — calling native C++ DLLs from C#
- C# & Rust via FFI — foreign function interface

Why it's useful: Teams can use the language best suited for each task. Legacy C++ code can be reused from new C# projects. Libraries built in F# can be consumed in VB.NET.

6. OOP in C# — Four Pillars

1. **Encapsulation** — hiding internal data, exposing only what's needed.

csharp

```
class BankAccount {  
    private double balance;           // hidden  
    public void Deposit(double amt) { balance += amt; }  
    public double GetBalance()        { return balance; }  
}
```

2. **Inheritance** — child class acquires members of parent class.

csharp

```
class Animal { public void Eat() { Console.WriteLine("Eating"); } }  
class Dog : Animal { public void Bark() { Console.WriteLine("Woof!"); } }  
  
Dog d = new Dog();  
d.Eat();    // inherited from Animal  
d.Bark();   // own method
```

3. **Polymorphism** — same method name, different behavior.

csharp

```
// Overloading (compile-time)  
int Add(int a, int b) { return a + b; }  
double Add(double a, double b) { return a + b; }  
  
// Overriding (runtime)  
class Animal { public virtual void Sound() { Console.WriteLine("..."); } }  
class Cat : Animal { public override void Sound() { Console.WriteLine("Meow"); } }  
class Dog : Animal { public override void Sound() { Console.WriteLine("Woof"); } }  
  
Animal a = new Dog();  
a.Sound(); // "Woof" - runtime decides based on actual object
```

4. Abstraction — hiding implementation, showing only the interface.

csharp

```
abstract class Shape {  
    public abstract double Area();    // no body — must be implemented  
}  
  
class Circle : Shape {  
    double r;  
    public Circle(double r) { this.r = r; }  
    public override double Area() { return 3.14 * r * r; }  
}
```

7. Common Console Methods

The `Console` class is in the `System` namespace. Key methods:

```
csharp

// Console.Write() - prints without newline
Console.Write("Hello ");
Console.Write("World"); // Output: Hello World (on same line)

// Console.WriteLine() - prints with newline
Console.WriteLine("Hello");
Console.WriteLine("World"); // Output: Hello (then) World on next line

// Console.ReadLine() - reads a full line of text input, returns string
string name = Console.ReadLine();

// Console.Read() - reads a single character, returns its ASCII int value
int ch = Console.Read();

// Console.ReadKey() - reads a single key press (no Enter needed)
ConsoleKeyInfo key = Console.ReadKey();
Console.WriteLine(key.Key); // e.g., "A"

// Console.Clear() - clears the console screen
Console.Clear();

// Formatted output with interpolation
string n = "Alice"; int age = 21;
Console.WriteLine($"Name: {n}, Age: {age}"); // Name: Alice, Age: 21
```

Method	Returns	Use
Write()	void	Print without newline
WriteLine()	void	Print with newline
ReadLine()	string	Read full line input
Read()	int	Read one character (ASCII)
ReadKey()	ConsoleKeyInfo	Read single key press
Clear()	void	Clear the screen

8. Identifiers and Keywords with Code Examples

Identifiers are names you give to variables, methods, classes, etc.

Rules:

- Must start with a letter or underscore `_`
- Can contain letters, digits, underscores
- Cannot be a C# keyword
- Case-sensitive (`age` and `Age` are different)

csharp

```
// Valid identifiers
int age = 25;
string _name = "Alice";
double totalAmount = 100.50;
int student1 = 10;

// Invalid identifiers
// int 1student = 10;    ✗ starts with digit
// int class = 5;        ✗ 'class' is a keyword
// int my-var = 3;       ✗ hyphen not allowed
```

Keywords are reserved words with special meaning to the compiler. You cannot use them as identifiers.

csharp



```
// Data type keywords
int age = 20;
double price = 99.99;
bool isActive = true;
string name = "Bob";

// Control flow keywords
if (age >= 18) {
    Console.WriteLine("Adult");
} else {
    Console.WriteLine("Minor");
}

for (int i = 0; i < 5; i++) { }
while (true) { break; }

// OOP keywords
class Student { }           // class definition
static void Main() { }     // static method
public int Id { get; set; } // access modifier

// Exception keywords
try { } catch (Exception e) { } finally { }

// Other
int[] arr = new int[5];     // new
this.name = name;          // this
```

Common keywords grouped: `if, else, switch, for, while, do, break, continue, return` (control flow) | `int, string, bool, double, char, decimal` (types) | `class, struct, enum, interface, delegate` (type declarations) | `public, private, protected, static, readonly, const` (modifiers) | `new, this, base, null, true, false` (others).

9. Data Types in Detail

(Already partially covered in L3 of the document, here's the full picture)

Value Types:

Type	Size	Range	Example
sbyte	1 byte	-128 to 127	sbyte x = -10;
byte	1 byte	0 to 255	byte b = 200;
short	2 bytes	-32,768 to 32,767	short s = 1000;
int	4 bytes	±2.1 billion	int n = 5000;
long	8 bytes	±9.2 quintillion	long l = 9999L;
float	4 bytes	±3.4×10 ³⁸ (7 digits)	float f = 3.14f;
double	8 bytes	±1.7×10 ³⁰⁸ (15-16 digits)	double d = 3.14;
decimal	16 bytes	±7.9×10 ²⁸ (28-29 digits)	decimal m = 99.99m;
char	2 bytes	Single Unicode char	char c = 'A';
bool	1 byte	true / false	bool b = true;

csharp

```
// Suffixes matter!  
float f = 3.14f;      // f suffix required  
double d = 3.14;      // default for decimals  
decimal m = 99.99m;   // m suffix required  
long l = 123456789L;  // L suffix for long
```

Reference Types:

csharp

```
string name = "Alice";      // immutable string  
object obj = 42;            // base of all types  
dynamic dyn = "Hello";     // runtime-resolved  
int[] arr = {1, 2, 3};     // array
```

Special: var (implicitly typed)

csharp

```
var x = 10;                // compiler infers int  
var name = "Bob";         // compiler infers string  
// Type is fixed at compile time – NOT dynamic
```


10. Operators and Their Types in Detail

This is essentially question 1 expanded with more depth. Here's the complete picture with all operator types:

Increment/Decrement (often missed!):

```
csharp

int a = 5;
Console.WriteLine(a++); // prints 5, THEN increments (post-increment)
Console.WriteLine(++a); // increments FIRST, then prints 7 (pre-increment)
Console.WriteLine(a--); // prints 7, THEN decrements
Console.WriteLine(--a); // decrements FIRST, then prints 5
```

Conditional / Ternary:

```
csharp

int marks = 75;
string grade = (marks >= 40) ? "Pass" : "Fail";
```

Null-related operators (very useful in C#):

```
csharp

string name = null;
Console.WriteLine(name ?? "Unknown"); // ?? null coalescing: "Unknown"

name ??= "Default"; // ??= assigns only if null
Console.WriteLine(name); // "Default"

// ?. null-conditional operator (safe navigation)
string s = null;
Console.WriteLine(s?.Length); // null instead of NullReferenceException
```